

Yiyuan Li

Singapore | yy@eyuan.me | +65 8952 3447 | eyuan.me | linkedin.com/in/yiyuanl | github.com/yuann3

Summary

Software engineer building AI agent infrastructure: agent tools and the sandboxing around them, evals, and the quality layer under inference routing that keeps multi-tenant agent platforms reliable. I anchor my work in the fundamentals of software engineering, which is why my side projects rebuild the classics: a SQLite engine from scratch, a Redis server, a GPU shader renderer that runs inside Emacs buffers, and an actor-based multi-agent runtime in Rust. 2+ years of experience, mostly in Rust, C, and TypeScript, with Python and Go when the job calls for it. I'm happiest figuring out how something works one layer deeper than I need to. I think computers and Emacs are cool, and so are house music and movies.

Technical Skills

Languages: Rust, C, C++, TypeScript, Python, Go (Golang), SQL, Emacs Lisp, GLSL, Bash

AI Agents & Inference: MCP, OAuth 2.1, Tool Design & Approval Gates, LLM-as-Judge & Probe Evals, Inference Provider Routing, Streaming & Cancellation, Mastra AI, RAG

Frameworks & Libraries: Tokio, wgpu, Bun, Hono, React, Vue 3, FastAPI, Flask, Node.js, Playwright

Databases & Infra: PostgreSQL (RLS, LISTEN/NOTIFY), pg-boss, Redis, SQLite, ChromaDB, Supabase, AWS, Docker, Turborepo, Git, Linux, CI/CD

Concepts: Agent Runtimes, Multi-Tenancy, Actor Model, Distributed Systems, Concurrency, Database Internals, GPU Programming, FFI/Dynamic Modules, TDD

Experience

Software Engineer, Voltade Pte. Ltd. – Singapore

Jun 2026 – Present

- Built the inference-quality layer under Volty's multi-provider LLM routing: detectors score every production reply for broken output, hosts that underperform their peers are quarantined, and a streaming breaker retries looping replies on another host
- Built the eval tooling around it: a probe harness that tests inference endpoints on tool calls, structured JSON, and long context against a reference endpoint, plus the LLM-judge eval behind smart inbox filters
- Designed and built agent tools: the first version of org MCP servers (OAuth 2.1, per-tool approval, hash-pinned definitions), PDF operations, and web fetch behind an SSRF guard with DNS pinning
- Split agent turns into foreground and background concurrency tiers, streamed tokens to the browser over Postgres NOTIFY and SSE with tenant scoping that fails closed, and made Stop a hard abort of in-flight tool calls
- Tech Stack: TypeScript, Bun, Hono, PostgreSQL (RLS), pg-boss, React 19, MCP, Playwright, AWS

Software Engineer – AI, ClassDo Pte. Ltd. – Singapore

Jan 2026 – May 2026

- Designed and built a multi-agent AI curriculum platform from scratch with a small team, and owned the architecture for how agents coordinate, validate, and hand off work
- Built the backend in a Turborepo monorepo: Hono middleware with JWT auth, RBAC, a typed RPC client, and 809+ tests across 59 files, deployed on AWS
- Integrated Singapore's SkillsFuture Skills Framework into Supabase (PostgreSQL), seeding 269K rows across 10 tables with multi-country schema support
- Built the Vue 3 frontend with Vike SSR, Pinia, and PrimeVue, with Storybook for the design system
- Tech Stack: TypeScript, Mastra AI, Hono, Vue 3, Supabase, PostgreSQL, AWS, Turborepo, Vitest, Zod

Software Engineering Intern, Newcastle Australia IHE Pte. Ltd. – Singapore

Jan 2025 – Sep 2025

- Built a full-stack RAG learning platform with FastAPI, React 19, and Ollama, giving 500+ users question answering over their course documents
- Built the document pipeline on LlamaParse with ChromaDB vector search, cutting query response time by 40%
- Built OAuth 2.0 authentication and REST API endpoints handling 1,000+ daily requests
- Tech Stack: Python, TypeScript, React 19, Next.js, FastAPI, Flask, Tailwind CSS, JWT, ChromaDB

Technical Leader, Data Structures & Algorithms, University of Newcastle

Feb 2024 – Nov 2024

- Led algorithm workshops on dynamic programming state transitions, graph traversal edge cases, and backtracking with pruning
- Wrote teaching material for the hard parts: B-tree rebalancing, amortized complexity, and off-by-one errors in binary search

Projects

MRA: Multi-Agent Runtime for Rust

[View on GitHub](#) ↗

- Designed an Erlang/OTP-inspired agent runtime where each AI agent runs as a Tokio actor with bounded mpsc mailboxes, zero shared mutable state, and transparent restarts via ArcSwap mailbox slots
- Implemented a supervisor with OneForOne/OneForAll restart strategies, exponential backoff, hang detection, and peer injection – agents survive crashes without callers noticing
- Technologies: Rust, Tokio, async/await, Actor Model, Serde, Figment, OpenRouter API

Emaclaude: Claude Code Orchestrator

[View on GitHub](#) ↗

- Built a 3-agent coding workflow (planner, coder, reviewer) inside Doom Emacs split buffers, coordinated by a ~130-line Rust CLI and an Elisp orchestrator piping JSON state and effects
- Implemented a review loop with double-confirmation passes to catch hallucinated approvals, plus inline diff comments and one-command PR creation via Magit and gh CLI
- Technologies: Rust, Emacs Lisp, Claude Code CLI, Magit, Serde, Clap, GitHub CLI

ShaderView: GLSL Shader Engine for Emacs

[View on GitHub](#) ↗

- Built a GPU-accelerated GLSL shader renderer that runs inside Emacs buffers using wgpu (Vulkan/Metal), with hot-reload on save and Shadertoy-compatible uniforms (iResolution, iTime, iMouse)
- Wrote a Rust dynamic module (cdylib FFI) bridging Emacs Lisp to a pure-Rust GPU core, supporting concurrent shader sessions, PNG export, and configurable frame rates
- Technologies: Rust, wgpu, Emacs Lisp, GLSL, FFI/Dynamic Modules, Vulkan, Metal

RayRust: CPU/GPU Path Tracer

[View on GitHub](#) ↗

- Built a Monte Carlo path tracer in Rust with Lambertian, metal, and dielectric (glass) materials, and a camera with depth of field and adjustable field of view
- Added a GPU backend on wgpu with WGSL shaders behind a feature flag, so the same scenes render on Vulkan, Metal, or DirectX 12, with the CPU renderer as fallback
- Technologies: Rust, wgpu, WGSL, GPU Compute, Monte Carlo Sampling

Sequel: SQLite Database Engine

[View on GitHub](#) ↗

- Built a SQLite-compatible database engine from scratch in Rust, implementing B-tree data structures and binary file parsing to read .db files without external dependencies
- Implemented SQL query execution including SELECT with WHERE clauses, COUNT aggregation, and index optimization achieving 3x faster lookups through B-tree traversal
- Technologies: Rust, B-tree, Binary Parsing, Database Internals, File I/O

Rego: Redis Server Implementation

[View on GitHub](#) ↗

- Implemented Redis-compatible server in Go with core commands (GET, SET, EXPIRE) and WAIT replication, passing 100% of Codecrafters integration tests
- Optimized memory allocation using slice pooling, reducing garbage collection overhead and improving throughput by 30%
- Technologies: Go (Golang), TCP/IP Networking, Concurrency, Memory Optimization

Ruskey: Programming Language Interpreter

[View on GitHub](#) ↗

- Developed a complete interpreter for Monkey language in Rust: lexer, recursive descent parser, AST, and tree-walking evaluator
- Implemented closures, first-class functions, and hash maps using Test-Driven Development (TDD) methodology
- Technologies: Rust, Compiler Design, Abstract Syntax Trees (AST), TDD

Education

Singapore University of Technology and Design (SUTD), 42 Singapore Computer Science

Nov 2025 – Sep 2026

- Core Curriculum: Peer-to-peer software engineering program focused on C, Unix, and systems programming
- Completed 16+ projects: libft, ft_printf, get_next_line, push_swap, pipex, Born2beRoot

The University of Newcastle, Bachelor of Information Technology

Jan 2024 – Sep 2025

- High Distinction: Object-Oriented Programming (OOP)
- Distinction: Data Structures and Algorithms (DSA), Advanced Database Systems